Journal of Computational Science 95 (2026) 102811

Contents lists available at ScienceDirect

Journal of Computational Science

journal homepage: www.elsevier.com/locate/jocs

dpGMM: A new R package for efficient and robust Gaussian mixture modeling of 1D and 2D data

Joanna Zyla [a,1], Kamila Szumala [a,1], Andrzej Polanski [b], Joanna Polanska [a], Michal Marczyk [a,c,*]

[a] Department of Data Science and Engineering, Silesian University of Technology, Gliwice, Poland
[b] Department of Computer Graphics, Vision and Digital Systems, Silesian University of Technology, Gliwice, Poland
[c] Breast Medical Oncology, Yale Cancer Center, Yale School of Medicine, New Haven, CT, USA

ARTICLE INFO

Keywords:
Gaussian mixture model
Clustering
Density estimation
Dynamic programming
R package

ABSTRACT

Gaussian Mixture Modeling (GMM) is a powerful clustering and density estimation method with various applications in data analysis. We introduce an R package, *dpGMM*, a complete set of tools/procedures to analyze 1D or 2D data (binned or continuous), including the most efficient existing solutions to problems of fitting GMM to data by the recursive expectation-maximization (EM) algorithm. The effectiveness of the *dpGMM* package comes from leveraging the power of EM recursions by: (i) precise choice of the initial mixture parameters obtained with the use of the dynamic programming-based partition of data, and (ii) augmenting each M step with additional conditions aimed at preventing instability/divergence and accelerating the rate of convergence of iterations. *dpGMM* is implemented as a wrapper that allows for searching the best decomposition in the scenario with an unknown number of Gaussian mixture components, by using various information criteria, as well as with a fixed number of components. We compared *dpGMM* with three other R packages using synthetic and real biological datasets, performing large-scale computations to assess the performance of these GMM implementations across various scenarios.

1. Introduction

Gaussian Mixture Modeling (GMM) is a powerful clustering procedure valued for its favorable statistical properties that enable soft clustering and facilitate the identification of the optimal number of clusters in the analyzed dataset. The GMM has wide application in many research fields, like speech recognition [1], video analysis [2], superpixel segmentation [3], network traffic analysis [4] and many others. With the development of high-throughput molecular biology, GMM also has a significant impact on biological and medical data analysis applications. Some examples of numerous applications of GMM in medical data analysis are the following. In mass spectrometry (MS), GMM was used for the decomposition of virtual MS data to estimate the location of signal peaks representing the mass and abundance of proteins and peptides [5]. This approach was further extended to efficiently analyze real data with hundreds or even thousands of peaks by sophisticated mass spectrum partitioning [6]. GMM was also used to detect stable isotopes in MALDI-TOF MS data [7] and applied in the decomposition of other high-throughput molecular biology data [8,9]. In [10], the authors used GMM for adaptive filtering of gene expression data measured with microarrays, while in [11], the authors created the package *GMMchi* for clustering of the same data type. Other examples of GMM applications are dedicated to the analysis of sequencing data. In [12], the authors applied GMM to detect nucleotide modification numbers in nanopore sequencing. In [13], the authors combined GMM with a deep autoencoder in the tool *scGMAI* and applied it to scRNA-sequencing data

* Correspondence to: Department of Data Science and Engineering, Silesian University of Technology, Akademicka 16, Gliwice 44-100, Poland.
E-mail address: michal.marczyk@polsl.pl (M. Marczyk).
[1] Equal contribution

https://doi.org/10.1016/j.jocs.2026.102811
Received 6 June 2025; Received in revised form 16 December 2025; Accepted 10 February 2026
Available online 11 February 2026

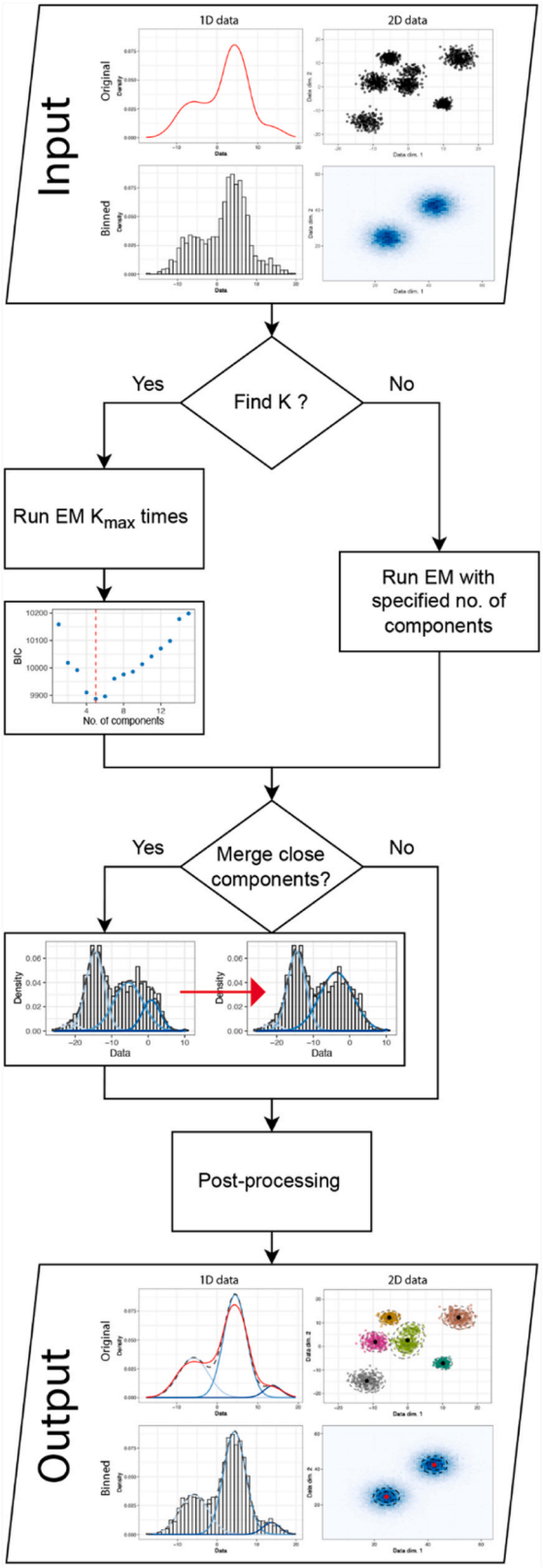

Fig. 1. Overview of *dpGMM* package functionality with possible types of input data. The first section relates to the possible input data, including 1D and 2D binned problems. Next, it shows the possibility of searching optimal number of components as well as solving fixed tasks. Further options for components merging and preprocessing are presented. The last part shows possible output visualizations of outcomes.

clustering. In [14], the authors demonstrated the advantages of GMM in detecting rare cell subtypes in mass cytometry data. Finally, GMM was applied sufficiently in biomedical image analysis [15,16].

All the above examples show that GMM is commonly used in bioinformatics, but not limited to, for data decomposition aimed at either clustering or density estimation. GMM implementations are widely available in the R and Python programming languages; however, they have multiple limitations, and their applications are often limited to 1D or 2D data without the option for binned data analysis. Moreover, some tools use random parameter initializations in the expectation–maximization (EM) procedure, which may lead to unstable results, or the optimal number of components search is not provided. To fill this gap, we propose a new, highly efficient, and robust GMM R package called *dpGMM*. The presented package can be applied to one-dimensional as well as two-dimensional data. *dpGMM* can deal with two types of variables/measurements: defined on continuous axes or binned, where the second one is commonly observed in spectra (e.g. mass spectrometry) or image analysis tasks (e.g. medical images). Additionally, our package is built as a wrapper so the decomposition can be done for a given number of clusters, or the optimal solution is searched using various criteria (e.g. AIC, AICc, or BIC). Finally, we reviewed existing R tools for GMM and showed the advantages of our approach.

2. Methods

2.1. dpGMM description and implementation

The proposed R package consists of several algorithms developed by the authors while experimenting with GMM using a wide range of biological data. The overview of *dpGMM* implementation and available features is presented in Fig. 1. Specifically, the method works on 1D or 2D signals (binned or continuous) and uses the EM algorithm to find the parameters of GMM. In general, the mixture of Gaussian components can be defined as:

$$f(\boldsymbol{x}) = \sum_{k=1}^{K} \alpha_k f_k(\boldsymbol{x}_k, \mu_k, \Sigma_k) \tag{1}$$

where K stands for the number of Gaussian components, α_k $(k = 1, 2, \ldots, K)$ are component weights that sum up to 1, f_k is the probability density function of the k-th Gaussian component, μ_k is the location of the k-th component (mean), and Σ_k is the covariance matrix of the k-th component. The *dpGMM* R package is freely available on GitHub https://github.com/ZAEDPolSl/dpGMM and CRAN https://cran.r-project.org/web/packages/dpGMM/index.html.

2.1.1. Dynamic programming-based initialization

The main advantage of *dpGMM* is a dynamic programming-based (DP-based) method for finding parameters of mixture distributions to initialize the EM [17] implemented for both 1D and 2D data. Setting initial values of the parameters of mixture distributions estimated using the recursive EM algorithm is very important to the overall quality of the fitted GMM. A problem is even more important in the case of heteroscedastic Gaussian mixtures with a large number of components. The algorithm for 1D data is based on DP partitioning of the range of observations into blocks to find the initial values of weights, means, and variances of Gaussian components. The blocks are searched using information derived from the data histogram to accelerate convergence in the Bellman recurrence. Details of the cost function and histogram partitioning are provided in [17], while the general concept is described in pseudo-code on Alg. 1.

Algorithm. 1. Dynamic programming for histogram partitioning

procedure GMM_DP_INIT(X, K)
  Compute histogram $(x, y) \leftarrow \text{Histogram}(X)$
  x: vector of bin centers (sorted)
  y: vector of histogram counts associated with x
  $s\_corr \leftarrow ((x[2] - x[1])^2)/12, \quad N \leftarrow \text{length}(x)$ ▷ histogram variance correction
  $aux\_mx \leftarrow \text{ComputeAuxMatrix}(x, y, s\_corr)$ ▷ calculate cost matrix
  Initialize p_opt_id as zero vector of size N
  Initialize opt_pals as zero array of size (K, N)
  for $kk = 1$ to N **do**
    $p\_opt\_idx[kk] \leftarrow \text{COST}(x[kk : N], y[kk : N], s\_corr)$
  end for
  for $k = 1$ to K **do** ▷ dynamic programming iterations
    for $i = 1$ to $N - k$ **do**
      for $j = i + 1$ to $N - k + 1$ **do**
        $p\_aux[j] \leftarrow aux\_mx[i, j] + p\_opt\_idx[j]$
      end for
      $p\_opt\_idx[i] \leftarrow \min(p\_aux[i + 1 : N - k + 1]), \quad opt\_pals[k, i] \leftarrow \text{argmin}(p\_aux[i + 1 : N - k + 1]) + i$
    end for
  end for
  Initialize opt_part as zero vector of size K
  $opt\_part[1] \leftarrow opt\_pals[K, 1]$ ▷ first boundary comes from last DP layer
  if $K > 1$ **then**
    for $k = K - 1$ down to 1 **do**
      $idx \leftarrow K - k + 1, \quad prev \leftarrow opt\_part[idx - 1], \quad opt\_part[idx] \leftarrow opt\_pals[k, prev]$ ▷ trace back decisions from DP
    end for
  end if
  $part\_idx \leftarrow [1, opt\_part, N + 1]$ ▷ add histogram boundaries
  Initialize $alpha\_ini$, μ_ini, σ_ini as zero vectors of size K
  for $kk = 1$ to K **do**
    $idx1 \leftarrow part\_idx[kk], \quad idx2 \leftarrow part\_idx[kk + 1] - 1$
    $xin \leftarrow x[idx1 : idx2], \quad yin \leftarrow y[idx1 : idx2], \quad w \leftarrow yin/\sum(yin)$
    $alpha\_ini[kk] \leftarrow \sum(yin)/\sum(y), \quad \mu\_ini[kk] \leftarrow \sum(xin * w), \quad var\_bin \leftarrow \sum((xin - \mu\_ini[kk])^2 * w)$
    if $var\_bin > s\_corr$ **then**
      $\sigma\_ini[kk] \leftarrow \sqrt{var\_bin - s\_corr}$
    else
      $\sigma\_ini[kk] \leftarrow \sqrt{var\_bin}$
    end if
  end for
  return $(alpha\_ini, \mu\_ini, \sigma\_ini)$
end procedure
function COMPUTEAUXMATRIX(x, y, s_corr) ▷ calculate costs for all segments
  $N \leftarrow \text{length}(x), \quad aux\_mx \leftarrow \text{zeros}(N, N)$
  for $kk = 1$ to $N - 1$ **do**
    for $jj = kk + 1$ to N **do**
      $aux\_mx[kk, jj] \leftarrow \text{COST}(x[kk : jj - 1], y[kk : jj - 1], s\_corr)$ ▷ segment cost
    end for
  end for
  return aux_mx
end function
function COST($xinvec, yinvec, s\_corr$) ▷ calculate quality for segment
  if $(xinvec[end] - xinvec[1]) \leq 0.1$ **or** $\sum(yinvec) \leq 1.0e - 3$ **then**
    return ∞
  else
    $w \leftarrow yinvec/\sum(yinvec), \quad var\_bin \leftarrow \sum((xinvec - \sum(xinvec * w))^2 * w)$
    if $var\_bin > s\_corr$ **then**
      return $(1 + \sqrt{var\_bin - s\_corr})/(\max(xinvec) - \min(xinvec))$
    else
      return ∞
    end if
  end if
end function

Output Definitions.

p_opt_idx**:** Vector contains optimal partial cumulative scores.

opt_pals**:** Matrix storing the optimal partition indices selected at each dynamic programming layer.

opt_part**:** Vector of length K that stores the final optimal partition boundaries.

Q**:** The value of the optimal score corresponding to the dynamic programming solution.

aux_mx**:** Auxiliary cost matrix is defined in order to efficiently compute optimal partitions for different numbers of blocks.

The algorithm worked well even with heteroscedastic Gaussian mixtures with many components and was more efficient in comparison to existing techniques for simulated and real datasets [17]. The initialization method was initially designed for 1D data, but in *dpGMM,* it is extended for 2D data analysis. For the 2D initialization the developed method consists of the following steps: (i) estimation of boundary distributions by summation of values over rows and columns; (ii) application of DP-based method to find initial condition for *K* components on two boundary distributions; (iii) aggregation of the two results to get all possible initial condition in 2D, which results in *K***K* components; (iv) iterative removal of single initial condition by maximizing log-likelihood until *K* initial conditions are left.

is controlled by the following formula:

$$\text{minvar} = \left(\frac{(\max - \min) * SW}{K}\right)^2 \tag{2}$$

where *max* and *min* are the maximum and minimum values of input data, *K* represents the total number of components, and *SW* is a hyperparameter controlled by the user (the higher the value, the higher the minimum variance of the Gaussian component is allowed). The EM algorithm together with "pin component" avoidance is described in pseudo-code on Alg. 2.

Algorithm. 2. EM algorithm for 1D Gaussian mixtures

procedure EM_ITER($X, \alpha, \mu, \sigma^2, Y, \varepsilon, max\_iter, SW$)
  $K \leftarrow \text{length}(\alpha)$
  $SW \leftarrow \left(\frac{\max(X) - \min(X)}{K} \cdot SW\right)^2$ ▷ "Pin components" protection
  $change \leftarrow \infty, \quad count \leftarrow 1$
  while $change > \varepsilon$ **and** $count < max\_iter$ **do**
    Save old parameter $old\_alpha \leftarrow \alpha, \quad old\_sig2 \leftarrow \sigma^2$
    for $i = 1$ **to** K **do**
      $f[i,] \leftarrow NormalPDF(X, \mu[i], \sqrt{\sigma^2[i]})$
    end for
    for $i = 1$ **to** K **do**
      $pk \leftarrow (\alpha[i] * f[i,] * Y) / \sum_{j=1}^{K} \alpha[j] * f[j,]$
      $\mu[i] \leftarrow \sum(pk * X) / \sum(pk)$
      $\sigma^2[i] \leftarrow \max\left(SW, \sum(pk * (X - \mu[i])^2) / \sum(pk)\right)$
      $\alpha[i] \leftarrow \sum(pk) / \sum(Y)$
    end for
    $change \leftarrow \sum |\alpha - old\_alpha| + \frac{1}{K} \sum |\sigma^2 - old\_sig2|$
    $count \leftarrow count + 1$
  end while
  Order components by μ to obtain $\mu_{\text{est}}, \ \sigma^2_{\text{est}}, \ \alpha_{\text{est}}$
  Compute log-likelihood: $logL \leftarrow \sum Y \cdot \log\left(\sum_{j=1}^{K} \alpha[j] f[j,]\right)$
  Compute information criterion (BIC, AIC, etc.): $crit$
  return $\{\mu_{\text{est}}, \sigma^2_{\text{est}}, \alpha_{\text{est}}, logL, crit\}$
end procedure

2.1.2. Pin component avoidance

Several additional features that increase the efficiency of model fitting are included in the EM iterations. The first one is avoiding divergence caused by "pin" components by introducing SW parameter. The "pin" components are the components with variances converging to 0 in the consecutive EM iterations. Often, this situation occurs when there are repeated (or very close) measurements in 1D data, such that their values are isolated from other measurements (visually, it is seen on a histogram as a high bar separated from other measurements). To avoid the problem, the minimum of the variance of each Gaussian component

2.1.3. EM stopping criteria

dpGMM implementation allows running GMM fitting with a user-specified number of components or searching for the best number of model components in a user-specified range using one of the available information criteria. The optimal number of components/clusters can be selected by several common information criteria: Akaike information criterion (AIC) [18], corrected Akaike information criterion (AICc) [19], Bayesian information criterion (BIC) [20], Integrated Completed Likelihood Bayesian information criterion (ICL-BIC) [21] and likelihood ratio test (LR). Further, the criterion for "early" stopping during GMM

estimation of mixtures of an unknown number of components, with multiple runs of EM recursions, was implemented. To obtain this goal, starting from *K= 2,* the likelihood ratio test (LRT) is performed between the solution with *K-1* components and the solution with *K* components. If the obtained p-value is lower than the significance level (α=0.05 by default), then searching is stopped, and the final number of model components is set to minimize the chosen information criterion in the range from 1 to *K*.

2.1.4. Components merging and post-processing

Due to different types of noises and artifacts in the data, a portion of the data points might be slightly shifted, even if the data comes from the same subpopulation. Thus, a procedure to merge similar Gaussian components in a close neighborhood is implemented (measured by standard deviation distance e.g. σ=1, between means). This parameter allows for merging components that are distant from each other by a given standard deviation. If set to 0, no merging is applied. Finally, scenarios for post-processing of the obtained clusters, where post-processing means using the obtained mixture parameters for tasks such as clustering (e.g., by using the maximum a posteriori rule), data filtering, or other tasks, are implemented. One of the biggest differences from other implementations is the distribution cut-off (thresholds) estimation. In the majority of solutions to solve this task, the maximum a posteriori probability (MAP) rule is used. In *dpGMM*, this technique is applied only when the Bayes–optimal decision rule [22], which is a quadratic equation reasoning, fails in the negative discriminant Δ. Thus, *dpGMM* uses combined thinking about decision boundaries. For two neighboring Gaussian components i and $i+1$, the Bayes–optimal boundary is obtained by equating the weighted component densities:

$$p_i(x) = \alpha_i \frac{1}{\sqrt{2\pi s_i}} \exp\left(-\frac{(x-\mu_i)^2}{2s_i^2}\right) \tag{3}$$

$$p_1(x) = p_2(x) \tag{4}$$

In the univariate case with unequal variances, Eqs. (3) and (4) lead to a quadratic equation in x, which solution is transformed to log space. The implemented procedure of threshold estimation has the following steps. First, A, B, and C parameters are calculated based on the model decomposition parameters of components i and $i+1$, as given by the expressions below:

$$A = \frac{1}{2*\sigma_i^2} - \frac{1}{2*\sigma_{i+1}^2} \tag{5}$$

$$B = \frac{\mu_{i+1}}{\sigma_{i+1}^2} - \frac{\mu_i}{\sigma_i^2} \tag{6}$$

$$C = \frac{\mu_i^2}{2*\sigma_i^2} - \frac{\mu_{i+1}^2}{2*\sigma_{i+1}^2} - \log\left(\frac{\alpha_i * \sigma_{i+1}}{\alpha_{i+1} * \sigma_i}\right) \tag{7a}$$

In the above, α_i is the weight of component i, μ_I is the mean of component I, and σ_i is the standard deviation of component i. Based on A, B, and C, several rules are applied to estimate the threshold between components i and $i+1$:

(i) If A and B are < 1e-10, the Gaussians are the same and no threshold is given
(ii) If A< 1e-10 but B >= 1e-10 the boundary reduces to a linear equation, giving x = –C/B
(iii) If both A and B > 1e-10, the discriminant Δ is calculated as follows and has two real solutions:

$$\Delta = B^2 - 4*A*C \tag{8a}$$

$$x_{1,2} = \frac{-B \pm \sqrt{\Delta}}{2A} \tag{9}$$

Finally, when the Bayes–optimal decision equation for two Gaussian components has no real solution (i.e., the quadratic discriminant is negative), *dpGMM* applies the MAP rule. The MAP classifier assigns an observation to the component with the largest posterior probability:

$$Class(x) = arg\max_i P(C_i|x) \propto arg\max_i \alpha_i p(x|C_i) \tag{10}$$

where $\alpha_i$is the mixing weight and p(x|C_i) is the Gaussian likelihood of component *i*. For two Gaussians, it leads to solving the task presented in Eq. 11 for which we choose the threshold as the point where one component first exceeds the other Eq. 12.

$$\alpha_i p_i(x) = \alpha_{i+1} p_{i+1}(x) \tag{11}$$

$$Choose\ x\ where\ \alpha_i p_i(x)\ is\ maximal\ relative\ to\ \alpha_{i+1} p_{i+1}(x) \tag{12}$$

From the obtained solutions, the final threshold is given only if it falls between the means of components i and $i+1$.

2.2. Other implementations in R

There are several packages in R implementing GMM clustering. In the presented research, the three most commonly used packages were compared to *dpGMM*. The first one is *mclust,* which is dedicated to GMM clustering only [23]. This package allows for the decomposition of 1D and 2D data and uses initialization obtained by hierarchical clustering. In the second package *ClusteR*, initial conditions can be set at random or by k-means. The package includes several clustering methods other than GMM, e.g., k-means [24]. The last tested package *mixtools* [25] is not limited to the Gaussian distribution but to a variety of finite mixture models with random initialization. All of the tested solutions allow for analyzing 1D and 2D data, yet none of them take into consideration binned representations of data. Moreover, the majority of them are not wrapper solutions that are easy to use. A comprehensive comparison of all features available in the above-mentioned packages and *dpGMM* is presented **in** Supplementary Table 1. To run the testing process for *mclust*, *ClusterR*, and *mixtools,* the *opGMMassessment* R package was used as a wrapper function [26] and the BIC was set as a criterion to select the best number of clusters. In this study, we compared dpGMM only to R packages. Other languages' GMM implementations contain similar features to the one that we analyzed. For example, *scikit-learn* in Python offers GMM with random and k-means initialization like *ClusteR*. Therefore, other solutions were not analyzed, since observed discrepancies would largely reflect differences in environments, rather than the algorithmic innovations

2.3. Testing procedure

2.3.1. Datasets

2.3.1.1. Simulation datasets. The synthetic data were generated in three scenarios: (i) with controlled overlap, (ii) without controlling overlap, and (iii) with controlled overlap and a constant number of Gaussian components of large sizes. The definition of the overlap coefficient, *OV*, between two Gaussians with parameters μ_1, σ_1 and μ_2, σ_2 was proposed in [17], and it follows:

$$OV = \exp\left(-\frac{|\mu_1 - \mu_2|}{2\sqrt{\sigma_1^2 + \sigma_2^2}}\right) \tag{7b}$$

In this scenario of generating data with controlled overlap, the 2100 different mixtures of normal distributions were created with the following parameters:

(i) the number of components is drawn uniformly from the range < 2;8 > ,

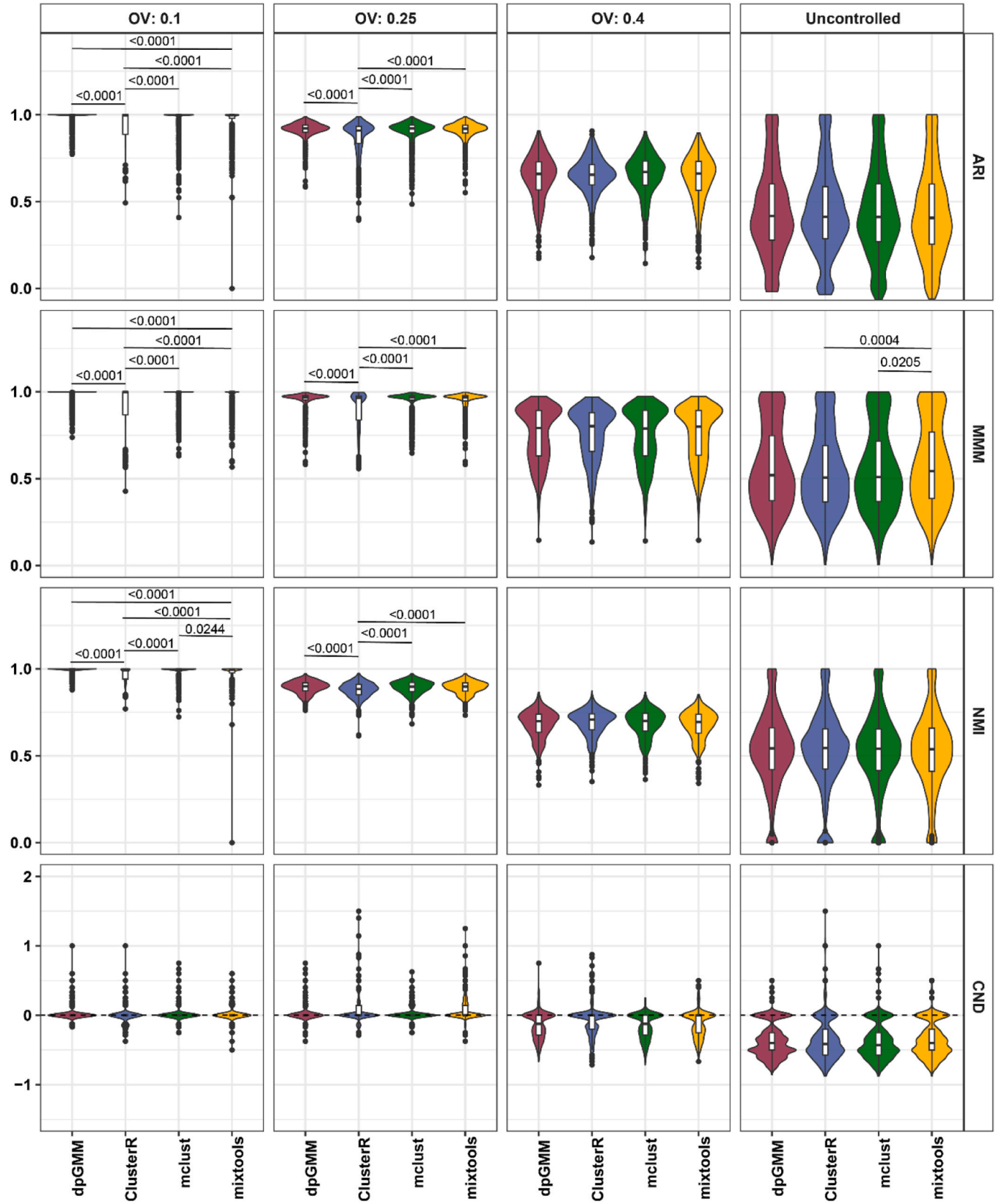

Fig. 2. Clustering performance metrics regarding the first and second scenarios of dataset generation across the tested R implementations. The statistical inference was performed using a pairwise Wilcoxon post-hoc test and is marked on the figure only if significant results were observed (α=0.05).

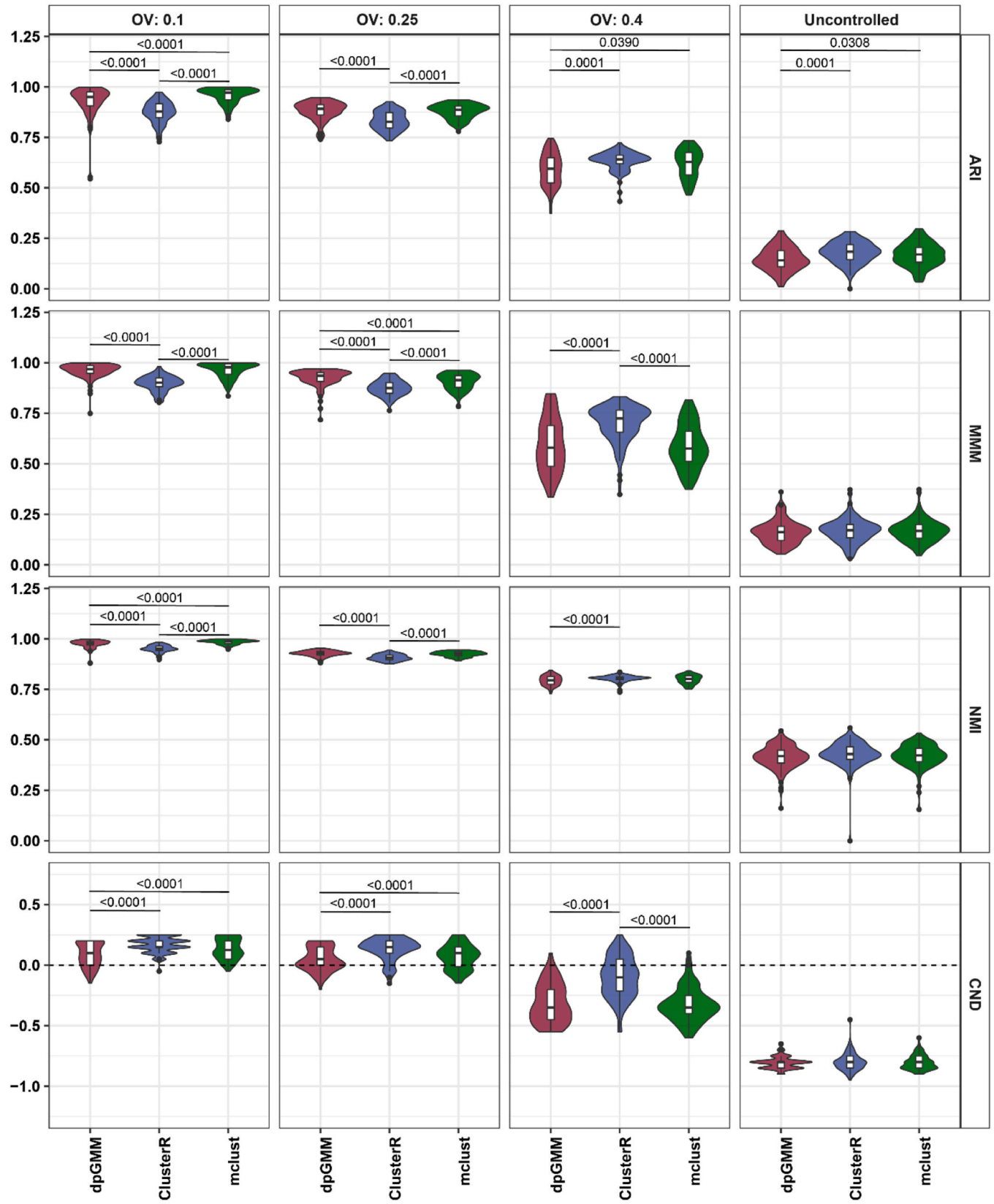

Fig. 3. Clustering performance metrics regarding the third scenario of dataset generation, where the expected cluster number equals 20. The statistical inference was performed using a pairwise Wilcoxon post-hoc test and is marked on the figure only if significant results were observed (α=0.05).

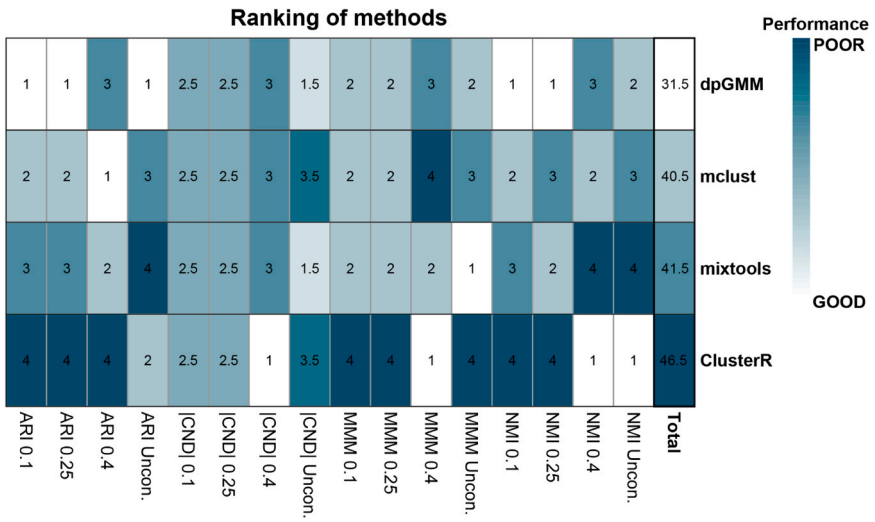

Fig. 4. Heatmap of ranking values for clustering performance metrics calculated based on median values. The lower the value, the better the outcome. Ranking was based on results from the first and second scenarios of dataset generation.

(ii) standard deviation (SD) of distribution components is drawn from range $< 0.1;1.0 >$ (uniform probability),
(iii) Gaussian mixture model weights are generated separately in the range 0.2–1 (with uniform probability) and then normalized to sum up to 1,
(iv) three different values of the overlap coefficient are assumed, OV= 0.1, OV= 0.25, and OV= 0.4, for each of 700 different mixtures of normal distributions,
(v) 1500 measurements were simulated.

The second scenario assumes no control of overlap, which in this case is random. In this scenario, the 1500 different mixtures of normal distributions are created with the following parameters:

(i) component number (cluster number) from range$< 2;8 >$ (uniform probability),
(ii) mean value of distribution components from range$< -15;15 >$ (uniform probability),
(iii) standard deviation (SD) of distribution components from range $< 1;5 >$ (uniform probability),
(iv) Gaussian mixture model weights are drawn independently, with uniform probability, from the range 0–1, then normalized,
(v) 1500 measurements were simulated.

In the third scenario, 100 datasets with 5000 measurements with a constant number of expected components/clusters equal to 20 were generated. The scenario is the same as in (i) or (ii) except that the number of components is not drawn randomly but set equal to 20. Those synthetic datasets will represent large problems that occur in high-throughput molecular biology data. For each of the generated mixtures of normal distributions, the cluster for each measurement was known.

Lastly, more complex datasets for testing the memory and time efficiency were generated. Testing was performed for mixtures from 2 to 35 components, as GMM is not effective and not dedicated to model data with a large number of clusters [27,28]. For each expected number of clusters, 10 datasets were generated with a large sample size of N = 80, 000 and OV= 0.1. In total, 4340 synthetic datasets were created.

2.3.1.2. Real-world datasets. Next to synthetic sets, several real-world datasets from biological problems were analyzed. The first one consists of 4 different variables, including values of the forward scatter and cytological markers (Cda, CDb, CDc) from flow cytometry with fluorescence-activated cell sorting [29]. The second real dataset includes stained histological (HE) images for which the estimation of the background signal level was investigated. This step is necessary to select only the important region of interest for further analysis and to reduce computational time. The image of a breast cancer patient slide from The Cancer Genome Atlas (TCGA) was obtained through The Cancer Imaging Archive [30]. The third real dataset represents 2D gel electrophoresis images, and the problem of spot detection was investigated. The 2-dimensional Gaussian function approximates the shape of a protein spot observed on images from gel electrophoresis. After dividing the whole gel into smaller areas, a mixture model is fitted to detect the positions of proteins in the gel. Exemplary data were created by extracting a segment from GelA [31] and processing using the 2DGMMgel algorithm [32]. Finally, a high-dimensional single-cell RNA sequencing (scRNA-Seq) dataset comprising over 80,000 cells/samples was collected [33](GSE182109). Using the ssGSEA enrichment algorithm [34] and glioma-associated microglia and macrophage (GAMs) markers [35], the activity of biomarkers was quantified in each cell, with higher ssGSEA scores indicating GAM-like cells. Next, these ssGSEA vectors were analyzed using GMM implementations, and the proportion of GAM cells relative to other cell types was calculated for each cluster. It is expected that the last cluster should contain the highest proportion of GAM cells.

2.3.2. Performance evaluation

For each dataset, all tested algorithms (i.e. *dpGMM*, *ClusterR*, *mixtools,* and *mclust)* were run, and the following characteristics were tested: (i) clustering performance; (ii) Gaussian mixture distribution parameter estimation. As synthetic data has known original cluster labels, the clustering given by each tested algorithm was evaluated by four metrics of different backgrounds: (i) adjusted rand index (ARI – Counting pairs background) [36]; (ii) normalized mutual information (NMI – Information theory background) [37]; (iii) maximum-match measure (MMM – Set matching background) [38]; (iv) cluster number deviation (CND – Cluster number background) defined by the following formula:

$$CND = \frac{\#of\ observe\ clusters - \#of\ expected\ clusters}{\#of\ expected\ clusters} \tag{8b}$$

Next, for known mixture-model parameters, the fitted and original point estimators' discrepancy was investigated. Finally, the *mixtools* package accepts other initialization points as input, so for scenarios one and two, the DP-based initialization was applied, and the same

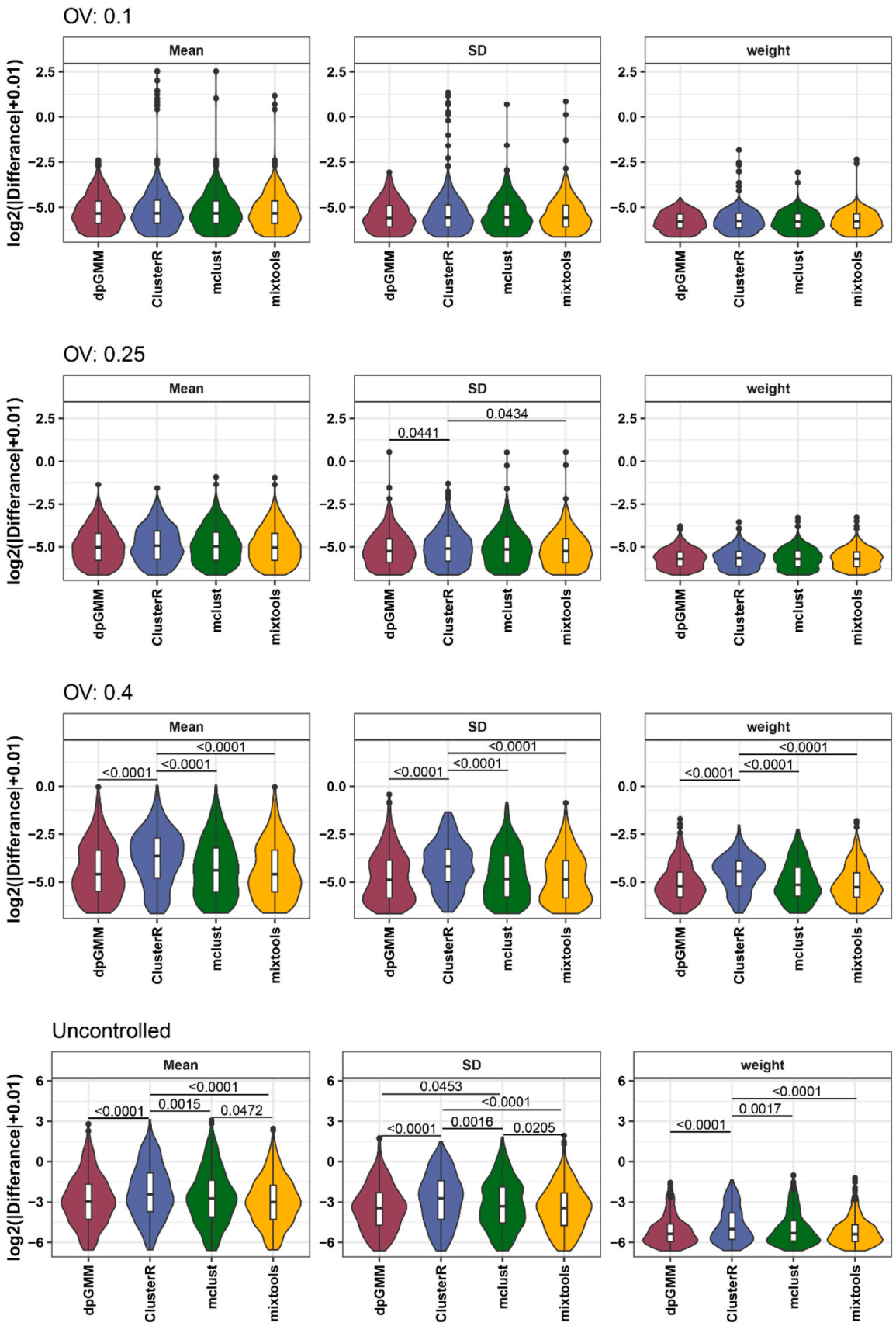

Fig. 5. Difference of mixture model parameters between original values and estimated in GMM decomposition presented in logarithmic scale with a shift of 0.01 to avoid infinite values (the lower the better). The statistical inference was performed using a pairwise Wilcoxon post-hoc test and is marked on the figure only if significant results were observed (α=0.05).

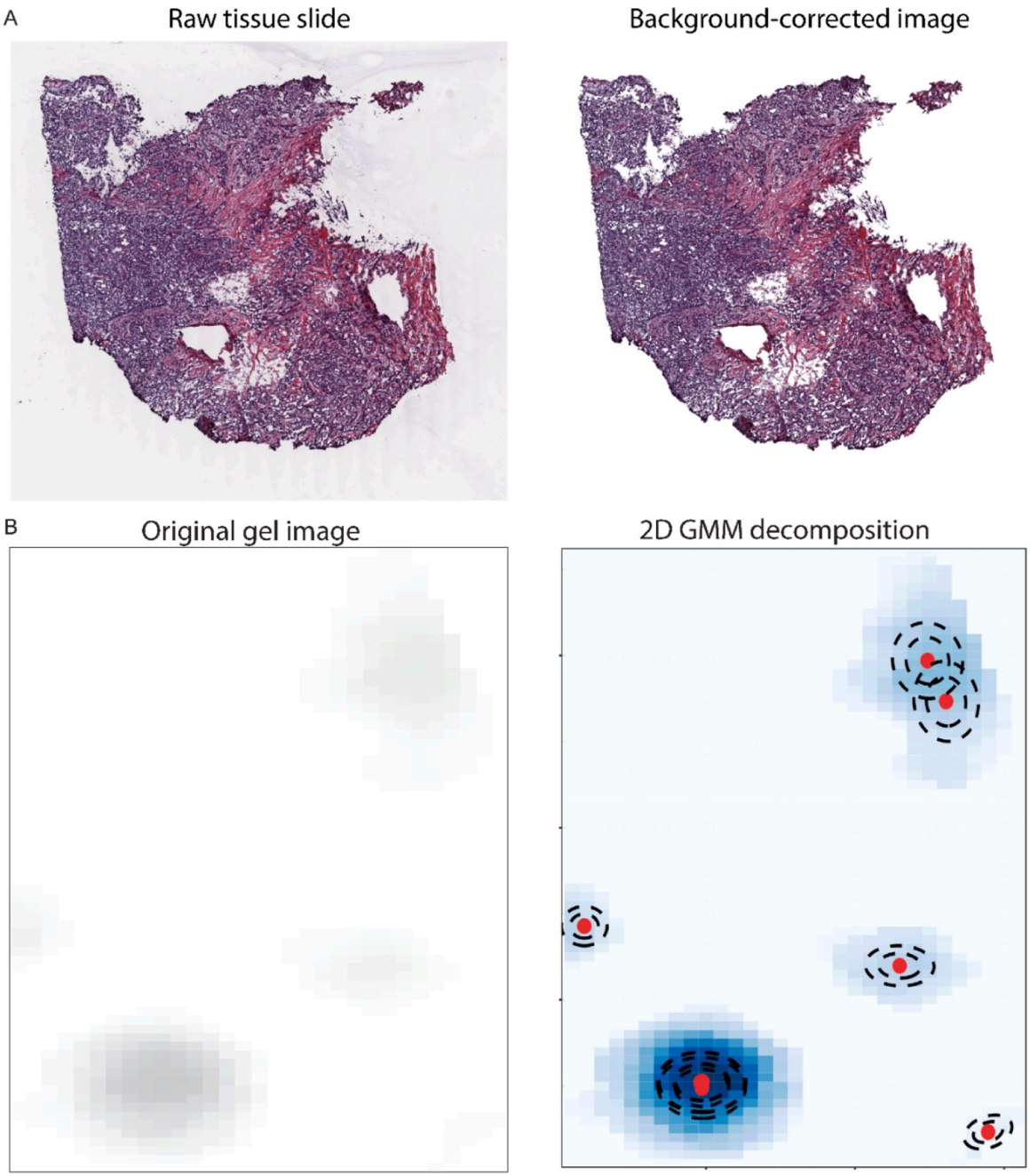

Fig. 6. Application of dpGMM to various real biological data. Panel A shows HE results for image background removal with the original HE image on the left and HE image after background noise removal on the right. Panel B shows a 2D gel image decomposition result for two different image segments with original gel images on the left and their decompositions on the right.

performance evaluation was performed.

Statistical analysis was performed to investigate differences between R packages at a 0.05 significance level. To quantify the distribution differences, the Kruskal-Wallis test with a pairwise Wilcoxon post-hoc test was performed. For the memory and time efficiency analysis, the 95 % confidence interval for the mean was calculated. The scheme of performed comparisons is presented in Supplementary Figure 1.

3. Results and discussion

3.1. Analysis of synthetic data

3.1.1. Clustering evaluation

For each algorithm, we evaluated synthetic data with known expected K components ranging from 2 to 8 and a case with K= 20. At first, we analyzed four clustering metrics (ARI, NMI, MMM, and Cluster number deviation) between each tested GMM implementation for a small number of components (2−8) and presented in Fig. 2. Moreover, to support the observed findings, the difference between *dpGMM* results and other implementations was calculated and presented in Supplementary Figure 2. As can be observed, the tested algorithms do not show superiority in terms of clustering performance metrics for large overlaps. Yet, for OV= 0.1, the proposed method is better than *ClusterR* (also observed for OV=0.25) and *mixtools*. Under uncontrolled overlap conditions, *mixtools* exhibited statistically significant superiority in the MMM metric compared to *mclust* and *ClusterR*, while the proposed *dpGMM* showed no significant advantage or disadvantage in this scenario. Next, we analyzed the case where the number of components

equals 20. Unfortunately, the computational time required by the *mixtools* package for a single dataset exceeded 14 days. Therefore, we excluded the *mixtools* algorithm from the large-data scenario, highlighting a notable limitation of this method. As shown in Fig. 3 (the difference between *dpGMM* and the other method is presented in Supplementary Figure 3), for overlap values of 0.1 and 0.25, *ClusterR* consistently yields the poorest results across all evaluated metrics. In contrast, for *OV* values of 0.4 and in scenarios with uncontrolled overlap, *ClusterR* demonstrates superior performance for ARI and MMM metrics. Under these conditions, *mclust* produces results comparable to those of the proposed *dpGMM*, which was only better in terms of CMD for low *OV*.

Further, we evaluated clustering performance concerning the expected number of clusters using Spearman correlation. All tested methods show a notable decline in performance with an increasing number of expected clusters, especially for the MMM metrics. This decline was more moderate for ARI and NMI at OV values of 0.1 and under uncontrolled overlap conditions (Supplementary Figure 4), which aligns with expected trends. Interestingly, for *OV* values of 0.25 and 0.4, the NMI metric improved as the number of clusters increased. Next, for the CND metric, we used the absolute values and constructed linear models to examine the relationship between the estimated number of clusters and the true number of components. Among the methods, the proposed *dpGMM* demonstrated the lowest variance in CMD (Fig. 2). Moreover, with increasing overlap and a larger number of clusters, in *dpGMM* estimation of the correct cluster number gets worse (Supplementary Figure 5). The same mechanism is observed for other tested methods. Further, we assessed the overall ranking of each method based on the median values of the performance metrics (Fig. 4). The proposed *dpGMM* achieved the best global performance, followed by *mixtools* and *mclust*, while *ClusterR* consistently showed the weakest performance.

3.1.2. Identification of the number of clusters

In the following step, we made an investigation of correctly detected clusters regarding the expected value (Supplementary Figure 6). It can be observed that *ClusterR* has the worst results for overlap 0.1 and 0.25. Next, the proposed *dpGMM* has the best performance for *OV* 0.1 and 0.25. When the overlap is not controlled, it has good performance for a small number of expected cluster numbers. Also, it is worth noticing that *dpGMM* has the best number of correctly detected components for expected $K = 20$ for OV 0.1 and 0.25. In the case of the other *OV*, all algorithms gave poor results for the large cluster problem, while for *mixtools* and $K = 20$, computational time was unreasonable for any practical usage. In Supplementary Figure 7**,** we presented a comparison of correctly identified models between methods. We can observe that almost 40 % of the tested models were common to all tested methods. Moreover, *dpGMM* is characterized by the largest amount of unique correct identifications for OV 0.1.

3.1.3. Mixture model parameters estimation

In the next step, we assessed how the obtained model parameters agree with the known ones. We selected only those results in which the number of detected clusters was in agreement with the expected number on the synthetic data. As presented in Fig. 5, *dpGMM* and *mixtools* have the best results among the tested methods (lowest median values). The *dpGMM* was significantly better than *ClusterR* for all model parameters for $OV = 0.4$ and uncontrolled ($p < 0.0001$). Similar results are observed for *mixtools* and *mclust* regarding their superiority to *ClusterR*. Finally, *dpGMM* has significantly better standard deviation estimation compared to *mclust*. Detailed p-values for all comparisons are included in Supplementary Figure 8.

3.1.4. Computational efficiency

The computational performance was investigated in terms of runtime and memory pick load (Supplementary Figure 9). The lowest computational time is observed for *ClusterR and mclust,* while mixtools shows a similar time to dpGMM. Yet, for *mixtools* we observe a high peak in computational time for a dataset with 17 clusters, and further computations were not possible to perform in a reasonable time. This drawback of *mixtools* is probably the result of no C+ + backend and the EM implementation not being optimized for the Gaussians with many components. In terms of memory load, the *ClusterR* package has the best performance, where memory usage only slightly increases with the cluster size and is at a maximum level of 25MB of RAM. For the investigated range, it can be observed that *dpGMM* and *mclust* have similar memory usage. With the increasing number of clusters, the *mclust* uses a larger portion of memory. Nevertheless, for both methods, the maximum is at 150MB of memory usage. Finally, when the number of clusters was set to 17, *mixtools* shows a similar memory load to *mclust* for $K = 35$, and probably is much larger for a higher number of clusters.

3.1.5. Testing dynamic programming-based initialization in mixtools

Finally, only *mixtools* package as input accepts initialization points given by the user. Thus, we compared the *dpGMM* with original *mixtools* and *mixtools* with DP-based initialization (mixtoool_DP). Results are presented in Supplementary Figure 10. It can be observed that for a small overlap of 0.1 for ARI, NMI, and MMM metrics, the performance of *mixtools* is improved when DP-based initialization is applied. Moreover, no statistically significant difference is observed between *mixtools* with DP and *dpGMM,* while such a difference is observed in favor of *dpGMM* compared with default *mixtools* initializations. Yet, promising results in favor of DP-based method may also improve the results on higher component overlaps.

3.2. Analysis of real biological data

The ability to analyze binned data greatly extends the practical applicability of *dpGMM*. Taking biological data as an example, in chromatography or mass spectrometry, a single measurement is seen as a function of ion intensity measured for compounds with different masses or other characteristics (1D binned data). For these data, an essential part of signal analysis is the detection and quantification of signal peaks, and using mixture models was found as a more efficient method of analysis than traditional solutions [6]. Another example is the problem of the detection and quantification of spots in gel images from electrophoresis experiments. Application of *dpGMM* allows finding spot positions with the surrounding boundaries and appropriate estimation of their quantities, mostly due to the detection of overlapping spots with higher sensitivity [32]. Finally, modeling the background can be realized by the use of a mixture of Gaussians for many different signals, including histopathological images of stained tissues [39].

To show how *dpGMM* works on real data, we applied it to several biological problems. As presented in Fig. 6A, *dpGMM* successfully removed background noise from the HE-stained breast cancer image collected from the TCGA database by thresholding on grayscale intensity level (1D binned data example). Next, we showed an application of *dpGMM* for spot finding in 2D gel images, which has binned information of protein position (Fig. 6B). In the selected fragment, we observe a few weak intensity but well-separated spots. The application of *dpGMM* allows for proper detection of protein positions. Next tested real data were taken from the flow cytometry experiment, which includes labeled classes. As can be observed in Supplementary Figure 11, *dpGMM* shows one of the best and most stable performances out of the tested methods. Finally, the high-resolution scRNA-Seq dataset (over 80k samples) was analyzed to detect GAM cells within the final clusters obtained from the decomposition. As shown in Supplementary Figure 12, Panel A, the decomposition produced by *mclust* is suboptimal, particularly in modeling the initial portion of the data. *Mixtools* yields numerous components that are either too broad or too narrow and often overlap, making the resulting clustering difficult to interpret. Decompositions for *ClusterR* and *dpGMM* appear more coherent. However, Supplementary

Figure 12, Panel B, demonstrates that *dpGMM* identifies two clusters with a high proportion of GAM cells, unlike ClusterR. This finding provides insight into heterogeneous cellular activity and reflects the more detailed decomposition of the right tail of the distribution.

4. Conclusions

In the presented study, we proposed a new R package called *dpGMM* and performed benchmarking of four different implementations of GMM in R. We showed that the proposed package performs similarly or better than other tested solutions using synthetic data of various characteristics as well as real data. It has a better estimation of Gaussian mixture-model parameters next to *mixtools*. However, *mixtools* implementation does not deal with searching a large number of clusters (K>17). Moreover, from our knowledge, *dpGMM* is the only implementation where 1D or 2D binned data (histograms or images) can also be modeled. Our package is built as a wrapper so the GMM decomposition can be automatically done for a given number of clusters, as well as for searching the optimal solution via various criteria. Additionally, several features that increase the efficiency of model fitting and prevent EM failure are implemented. Our package includes visualization of decompositions for both 1D and 2D data. All of those give users the possibility to better control the quality of Gaussian mixture decomposition and its usage in real applications.

Funding

This work has been supported by the Silesian University of Technology grant for maintaining and developing research potential for 2026 [KS, AP, JP], by the Rector's habilitation grant available under the Excellence Initiative – Research University program, Silesian University of Technology, grant no. 02/070/SDU/10–07-01 [JZ], and National Science Centre, Poland, grant no. 2023/50/E/NZ2/00583 [MM].

CRediT authorship contribution statement

Michal Marczyk: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization. **Joanna Polanska:** Writing – review & editing, Supervision, Funding acquisition. **Andrzej Polanski:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Funding acquisition, Conceptualization. **Kamila Szumala:** Visualization, Software, Investigation, Data curation. **Joanna Zyla:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper

Acknowledgments

Not applicable.

Appendix A. Supporting information

Supplementary data associated with this article can be found in the online version at doi:10.1016/j.jocs.2026.102811.

Data Availability

Data used in the presented study are available within the R package dpGMM (https://github.com/ZAEDPolSl/dpGMM), scRNA-Seq data under accession no. GSE182109 and EDOtrans (https://cran.r-project.org/web/packages/EDOtrans/index.html). Full synthetic data, HE images, and 2D gel image data are available upon reasonable request.

References

[1] M. Azam, N. Bouguila, Multivariate-bounded Gaussian mixture model with minimum message length criterion for model selection, Expert Syst. 38 (5) (2021) e12688.
[2] Y. Fan, G. Wen, D. Li, S. Qiu, M.D. Levine, F. Xiao, Video anomaly detection and localization via Gaussian mixture fully convolutional variational autoencoder, Comput. Vis. Image Underst. 195 (2020) 102920.
[3] Z. Ban, J. Liu, L. Cao, Superpixel segmentation Using Gaussian mixture model, IEEE Trans. Image Process (2018).
[4] R. Chapaneri, S. Shah, Multi-level Gaussian mixture modeling for detection of malicious network traffic, J. Supercomput. 77 (5) (2021) 4618–4638.
[5] M. Plechawska, J. Polańska, Simulation of the usage of Gaussian mixture models for the purpose of modelling virtual mass spectrometry data, Stud. Health Technol. Inf. 150 (2009) 804–808.
[6] A. Polański, M. Marczyk, M. Pietrowska, P. Widlak, J. Polańska, Signal partitioning algorithm for highly efficient gaussian mixture modeling in mass spectrometry, PLoS ONE 10 (7) (2015) 1–19.
[7] J.C.G. Spainhour, M.G. Janech, V. Ramakrishnan, A study of the properties of Gaussian mixture model for stable isotope standard quantification in MALDI-TOF MS, Commun. Stat. Simul. Comput. 48 (6) (2019) 1637–1650.
[8] M. Wang, J.Y. Chen, A GMM-IG framework for selecting genes as expression panel biomarkers, Artif. Intell. Med. 48 (2) (2010) 75–82.
[9] A. Mirzal, Statistical analysis of microarray data clustering using NMF, spectral clustering, Kmeans, and GMM, IEEE/ACM Trans. Comput. Biol. Bioinform. 19 (2) (2022) 1173–1192.
[10] M. Marczyk, R. Jaksik, A. Polański, J. Polańska, Adaptive filtering of microarray gene expression data based on Gaussian mixture decomposition, BMC Bioinforma. 14 (art. 101) (2013) 1–12.
[11] T.-C. Liu, P.N. Kalugin, J.L. Wilding, W.F. Bodmer, GMMchi: gene expression clustering using Gaussian mixture modeling, BMC Bioinforma. 23 (1) (2022) 457.
[12] H. Ding, A.D. Bailey, M. Jain, H. Olsen, B. Paten, Gaussian mixture model-based unsupervised nucleotide modification number detection using nanopore-sequencing readouts, Bioinformatics 36 (19) (2020) 4928–4934.
[13] B. Yu, C. Chen, R. Qi, R. Zheng, P.J. Skillman-Lawrence, X. Wang, et al., scGMAI: a Gaussian mixture model for clustering single-cell RNA-Seq data based on deep autoencoder, Brief. Bioinforma. 22 (4) (2021) bbaa316.
[14] A. Suwalska, J. Polańska, GMM-based expanded feature space as a way to extract useful information for rare cell subtypes identification in single-cell mass cytometry, Int. J. Mol. Sci. 24 (18) (2023) 1–13.
[15] F. Binczyk, B. Stjelties, C. Weber, M. Goetz, K. Meier-Hein, H.-P. Meinzer, et al., MiMSeg - an algorithm for automated detection of tumor tissue on NMR apparent diffusion coefficient maps, Inf. Sci. 384 (2017) 235–248.
[16] F. Riaz, S. Rehman, M. Ajmal, R. Hafiz, A. Hassan, N.R. Aljohani, et al., Gaussian mixture model based probabilistic modeling of images for medical image segmentation, IEEE Access 8 (2020) 16846–16856.
[17] A. Polański, M. Marczyk, M. Pietrowska, P. Widlak, J. Polańska, Initializing the EM algorithm for univariate gaussian, multi-component, heteroscedastic mixture models by dynamic programming partitions, Int. J. Comput. Methods 15 (1) (2018) 1–21, art. 1850012.
[18] Akaike H., editor Information theory and an extension of the maximum likelihood principle. International Symposium on Information Theory, 2 nd, Tsahkadsor, Armenian SSR; 1973.
[19] D. Anderson, K. Burnham, Model selection and multi-model inference, 63, Springer-Verlag, Second NY, 2004, p. 10.
[20] G. Schwarz, Estimating the dimension of a model, Ann. Stat. 6 (2) (1978) 461–464.
[21] C. Biernacki, G. Celeux, G. Govaert, Assessing a mixture model for clustering with the integrated completed likelihood, IEEE Trans. Pattern Anal. Mach. Intell. 22 (7) (2000) 719–725.
[22] R.O. Duda, P.E. Hart, Pattern Classification, John Wiley & Sons, 2006.
[23] L. Scrucca, M. Fop, T.B. Murphy, A.E. Raftery, mclust 5: Clustering, Classification and Density Estimation Using Gaussian Finite Mixture Models, R. J. 8 (1) (2016) 289–317.
[24] Mouselimis L. ClusterR: Gaussian Mixture Models, K-Means, Mini-Batch-Kmeans, K-Medoids and Affinity Propagation Clustering. R package version 1.3.3 ed2024.
[25] T. Benaglia, D. Chauveau, D.R. Hunter, D.S. Young, mixtools: An R Package for Analyzing Mixture Models, J. Stat. Softw. 32 (6) (2009) 1–29.
[26] J. Lötsch, S. Malkusch, A. Ultsch, Comparative assessment of automated algorithms for the separation of one-dimensional Gaussian mixtures, Inform. Med. Unlocked 34 (2022) 101113.
[27] G.J. McLachlan, K.E. Basford, Mixture models. Inference and applications to clustering, Stat. Textb. Monogr. (1988).
[28] M.A.T. Figueiredo, A.K. Jain, Unsupervised learning of finite mixture models, IEEE Trans. Pattern Anal. Mach. Intell. 24 (3) (2002) 381–396.
[29] A. Ultsch, J. Lötsch, Euclidean distance-optimized data transformation for cluster analysis in biomedical data (EDOtrans), BMC Bioinforma. 23 (1) (2022) 233.
[30] K. Clark, B. Vendt, K. Smith, J. Freymann, J. Kirby, P. Koppel, et al., The cancer imaging archive (TCIA): maintaining and operating a public information repository, J. Digit. Imaging 26 (6) (2013) 1045–1057.

[31] B. Raman, A. Cheung, M.R. Marten, Quantitative comparison and evaluation of two commercially available, two-dimensional electrophoresis image analysis software packages, Z3 and Melanie, Electrophoresis 23 (14) (2002) 2194–2202.
[32] M. Marczyk, Mixture modeling of 2D gel electrophoresis spots enhances the performance of spot detection, IEEE Trans. NanoBiosci. 16 (2) (2017) 1–9.
[33] N. Abdelfattah, P. Kumar, C. Wang, J.-S. Leu, W.F. Flynn, R. Gao, et al., Single-cell analysis of human glioma and immune cells identifies S100A4 as an immunotherapy target, Nat. Commun. 13 (1) (2022) 767.
[34] D.A. Barbie, P. Tamayo, J.S. Boehm, S.Y. Kim, S.E. Moody, I.F. Dunn, et al., Systematic RNA interference reveals that oncogenic KRAS-driven cancers require TBK1, Nature 462 (7269) (2009) 108–112.
[35] P. Kaminska, P.L. Ovesen, M. Jakiel, T. Obrebski, V. Schmidt, M. Draminski, et al., SorLA restricts TNFα release from microglia to shape a glioma-supportive brain microenvironment, EMBO Rep. 25 (5) (2024) 2278–2305.
[36] W.M. Rand, Objective criteria for the evaluation of clustering methods, J. Am. Stat. Assoc. 66 (336) (1971) 846–850.
[37] A. Strehl, J. Ghosh, Cluster ensembles - a knowledge reuse framework for combining multiple partitions, J. Mach. Learn. Res. 3 (2002) 583–617.
[38] M. Meilă, Comparing clusterings: an axiomatic view. Proceedings of the 22nd international conference on Machine learning; Bonn, Association for Computing Machinery, Germany, 2005, pp. 577–584.
[39] M. Marczyk, A. Wrobel, J. Merta, J. Polanska, Post-Processing of Thresholding or Deep Learning Methods for Enhanced Tissue Segmentation of Whole-Slide Histopathological Images, in: In proceedings of the 18th International Joint Conference on Biomedical Engineering Systems and Technologies, 1, 2025, pp. 229–238.

Glossary

AIC: Akaike information criterion
AICc: Corrected Akaike information criterion
ARI: Adjusted Rand index
BIC: Bayesian information criterion
CND: Cluster number deviation
DP: Dynamic programming
EM: Expectation-Maximization
GAM: glioma-associated microglia and macrophage (cells)
GMM: Gaussian Mixture Modeling
HE: Stained histological images
ICL-BIC: Integrated completed likelihood Bayesian information criterion
LRT: Likelihood ratio test
MAP: Maximum a posteriori probability
MMM: Maximum-match measure
MS: Mass spectrometry
NMI: Normalized mutual information
OV: Overlap
scRNA-Seq: single-cell RNA sequencing
TCGA: The Cancer Genome Atlas